VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY

UNIVERSITY OF SCIENCE

FACULTY OF INFORMATION TECHNOLOGY

PROJECT REPORT

CS163

Group members:

1. 25125052 - Võ Thành Đạt
2. 25125055 - Phạm Huy Khánh
3. 25125068 - Lê Phi Truyền
4. 25125089 - Lê Trọng Đại Nghĩa

22nd March 2026

Contents

1 Summary **2**

2 Introduction **3**

3 Group Info **4**

4 Data Storage **5**

5 Architecture **6**

6 Implementation **8**

7 Features **9**

8 Conclusion **10**

1 Summary

This report presents the comprehensive design and implementation of the ***DSA Visualizer****, an interactive tool developed in* ***C++*** *using the* ***SFML*** *library. The project aims to enhance the understanding of complex algorithms through real-time visualization of four distinct areas:* ***Singly Linked Lists****,* ***Hash Tables****,* ***Tries (Prefix Trees)****, and* ***Minimum Spanning Trees (MST)****. Adhering to a strict architectural separation of concerns, the system isolates pure logic from graphical rendering, ensuring high maintainability and performance. By utilizing a frame-independent animation system and a centralized* ***ScreenManager****, the application provides a smooth, interactive experience for exploring data operations and graph algorithms. This summary highlights the design choices, the* ***technical*** *challenges of synchronizing states, and the final results achieved by the group.*

2 Introduction

In the field of computer science education, understanding complex data structures and algorithms (DSA) often poses a significant challenge for learners. Traditional teaching methods, which rely heavily on static diagrams and theoretical explanations, can make it difficult for students to fully grasp dynamic processes such as pointer manipulation, hashing, tree traversal, and graph optimization. As a result, there is a growing need for more interactive and intuitive learning tools that can bridge the gap between theory and practice.

Motivated by this challenge, this project presents the design and development of a **DSA Visualizer**, an interactive application built using **C++** and the **SFML (Simple and Fast Multimedia Library)** framework. The primary goal of this project is to enhance the learning experience by providing real-time visual representations of fundamental data structures and algorithms. By allowing users to observe how operations are executed step by step, the system transforms abstract concepts into tangible and engaging visual experiences.

The visualizer focuses on four key areas: **Singly Linked Lists**, **Hash Tables**, **Tries (Prefix Trees)**, and **Minimum Spanning Trees (MST)**. These structures were carefully selected due to their importance in both academic curricula and real-world applications. Through interactive simulations, users can perform operations such as insertion, deletion, searching, and traversal, while simultaneously observing how the underlying data structure evolves.

This project is important because it not only improves conceptual understanding but also promotes active learning. By integrating a clear architectural separation between core logic and graphical rendering, the system ensures maintainability, scalability, and performance efficiency. Ultimately, the DSA Visualizer aims to serve as a practical educational tool that empowers students to explore, experiment, and develop a deeper intuition for algorithmic thinking.

3 Group Info

4 Data Storage

5 Architecture

The architecture of the Data Structures and Algorithms (DSA) Visualizer project is structured hierarchically to enforce a strict separation of concerns among the core application loop, user interface states, graphical rendering, and the underlying mathematical data models. At the highest entry and management level, the execution delegates the primary event loop, comprising state updates and rendering, to an application core. This core relies on a scene manager that governs the overarching application state by maintaining a collection of scenes via unique smart pointers, facilitating dynamic transitions between discrete functional areas of the software. Concurrently, a resource manager employs the Singleton design pattern to centralize the memory management and distribution of graphical assets, such as fonts and textures.

To decouple user input from system execution, the architecture implements a robust Command Pattern layer. This mechanism revolves around an abstract interface defining a pure virtual execution method, which is subsequently realized by specialized command implementations. Scene commands leverage factories and lambda functions to instantiate new user interface states dynamically, while operation commands encapsulate specific algorithmic manipulations requested within the visualizer interfaces.

The graphical interface is managed by a distinct scene layer anchored by a foundational base interface. This base dictates a standard lifecycle requiring derived classes to process events, update logical states, and render graphical components. This hierarchy branches into navigational elements, such as menu scenes containing interactive callback-driven buttons, and complex visualization scenes. These abstract visualization scenes serve as templates that house shared user interface components and define abstract contracts for algorithmic operations like insertion, searching, deletion, and traversal. Concrete implementations for specific paradigms—such as linked lists, graphs, trees, and hash sets—inherit from this abstract scene to implement the distinct user interactions required for each structural type.

Bridging the interactive interfaces and the raw algorithms is the data model and visualizer layer. A base visualizer interface establishes the mandatory protocols for resetting state and

pushing graphical representations to the rendering window. Specific visualizer classes implement these protocols to translate raw data into dynamic on-screen components. At the foundational base of the entire architecture lie the pure data model files. These components encapsulate the intrinsic algorithmic logic of the data structures completely independently of the Simple and Fast Multimedia Library (SFML) or any graphical dependencies. The visualizers maintain references to these pure models, ensuring that the visual representation merely observes and reflects the state of the isolated algorithmic backend.

Given this architecture, it is worth considering how the system manages the temporal synchronization between the pure, instantaneous C++ data models and the sequential, frame-by-frame animations required by the visualizer layer. Furthermore, one might examine how the Command Pattern specifically handles the undoing of state-altering operation commands, and whether the purely decoupled nature of the data models requires an intermediary event-messaging system to notify the visualizers of rapid, deep-nested algorithmic state changes during complex operations like tree balancing.

6 Implementation

7 Features

8 Conclusion